Pune Institute of Computer Technology

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Department of Computer Engineering

AY : 2026-27

CNL MINI PROJECT REPORT

ON

“ Real-Time Network Intrusion Detection and Traffic Monitoring System ”

Submitted By

Roll No.	Name
31167	Ruchit Bhalerao
31181	Vishwaja Deshmukh

Class : TY-01 Batch : N1

Under the guidance of

Prof. A. D. Bundele

Index

1. Problem Statement

2. Objectives

3. Methodology / Approach (Architecture & Flow Diagrams)

4. Implementation

5. Results / Outputs

6. Conclusion

7. References

1. Problem Statement

Modern networks are continuously exposed to reconnaissance and attack patterns such as port scanning, ARP spoofing, abnormal traffic bursts, and connections from unauthorised devices. At the same time, administrators need to understand ordinary network behaviour — how much bandwidth is used, which protocols dominate, and which hosts are the busiest. Professional intrusion-detection and traffic-analysis tools exist, but they are often heavyweight, expensive, and opaque, which makes them unsuitable for learning the underlying networking concepts.

This mini project addresses the Computer Networks domain of Network Monitoring and Traffic Analysis. The problem is to design and implement a lightweight, transparent, and safe system that:

- captures and parses network packets (Ethernet, ARP, IP, TCP, UDP, ICMP) into a normalised form,
- detects common classes of suspicious activity using well-defined, configurable rules,
- measures live traffic — bandwidth, protocol distribution, and top talkers — and visualises it in real time, and
- stores the resulting security events and traffic metrics and exposes them through a read-only web dashboard and a live traffic portal, strictly on the local machine.

The system is built using only Python together with Scapy, Flask, and the standard-library sqlite3 module, so every layer — from packet parsing to detection to visualisation — can be read and understood, which also supports demonstration and viva discussion.

2. Objectives

- To capture packets from a live network interface (using Scapy) and, for safe demonstration, from a synthetic traffic generator, and to normalise each packet into a common PacketRecord structure.
- To detect five classes of possible suspicious activity: port scanning, traffic-rate anomalies, ARP anomalies (spoofing), unknown devices, and repeated connection attempts.
- To measure network traffic in real time — packets per second, bytes per second (bandwidth), protocol mix, and the busiest source hosts (top talkers).
- To persist alerts, observed devices, and per-second traffic metrics in a SQLite database, and to expose them through a read-only JSON API.
- To provide a minimal, real-time web interface: an alerts dashboard and a live traffic monitoring portal with bandwidth and protocol visualisation.
- To keep the system safe and ethical: it only observes traffic, never generates attacks or scans, and the web server binds to 127.0.0.1 (localhost) only.
- To keep the design modular, configuration-driven (all thresholds in a single config file), and verifiable through an automated test suite.

3. Methodology / Approach

The system follows a clear pipeline: a packet source feeds a parser, which produces normalised records; each record is processed by a detection engine (rule-based detectors with deduplication) and, in parallel, by a traffic meter; results are stored in SQLite and served over a read-only Flask JSON API to the web interface.

3.1 System Architecture

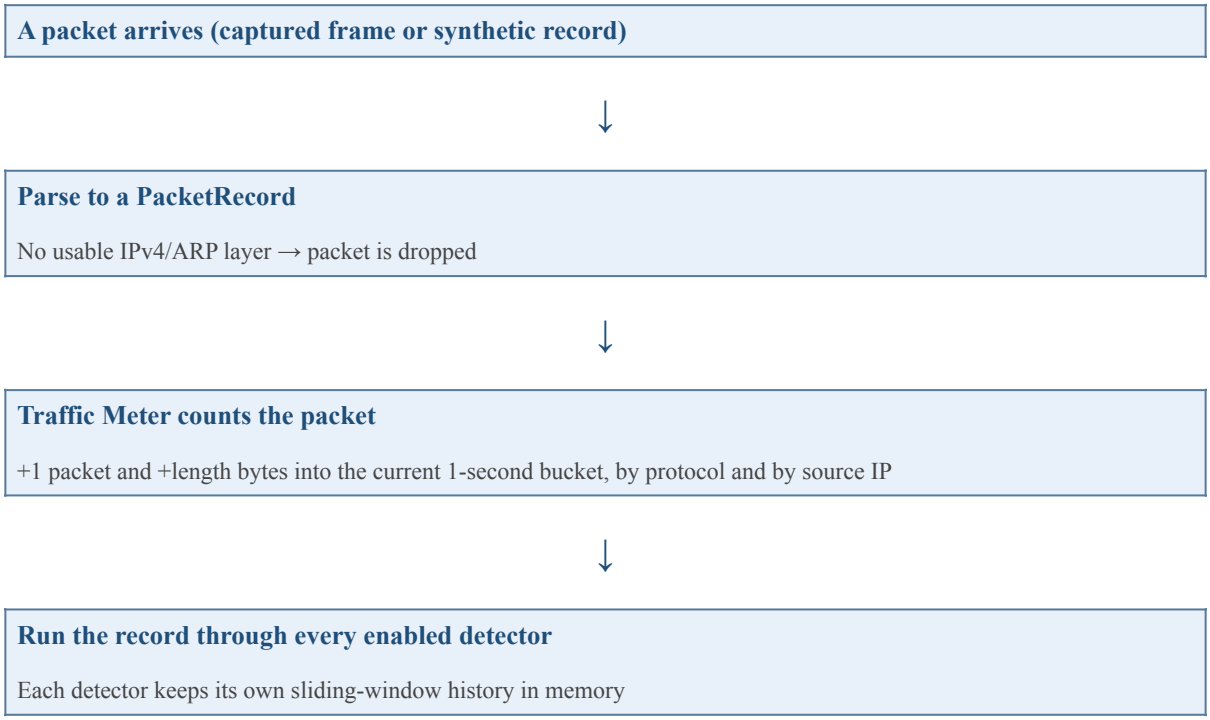
Packet Sources

Live capture via Scapy sniff() • Synthetic traffic feed (--live-demo)



Figure 1: System architecture / data-flow of the NIDS and live traffic portal.

3.2 Packet-Processing Flow



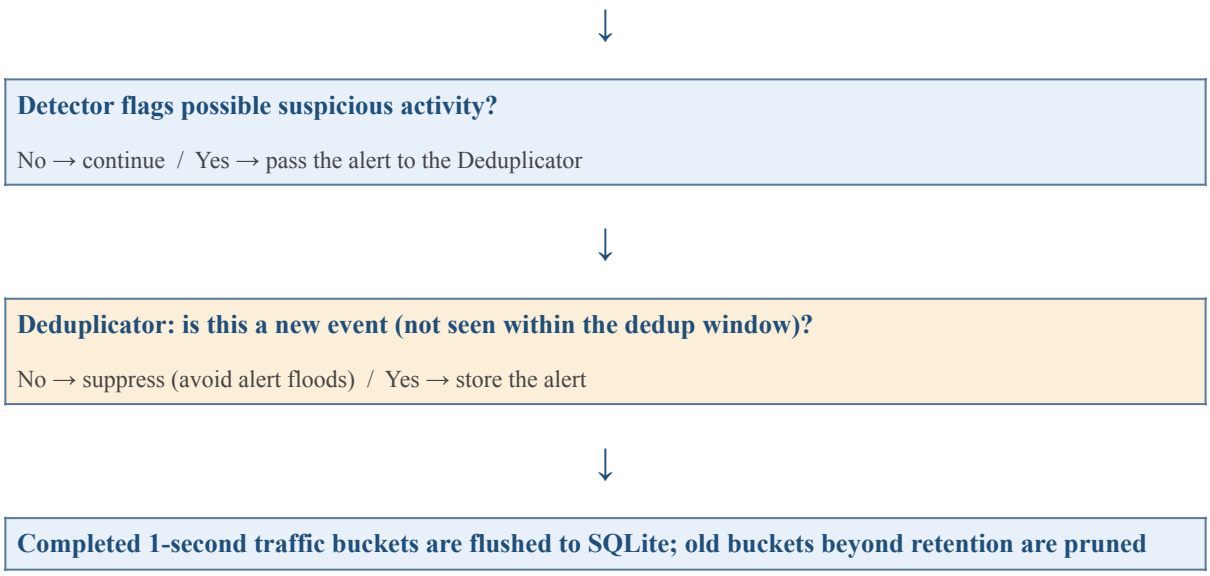


Figure 2: Per-packet processing and detection flow.

3.3 Networking Concepts Demonstrated

- **Layer 2 – Ethernet & ARP:** source/destination MAC addresses are read from frames; ARP sender IP–MAC bindings are tracked to detect ARP spoofing (one IP suddenly claiming a different MAC).
- **Layer 3 – IPv4:** source and destination IP addresses and the protocol field (TCP=6, UDP=17, ICMP=1) are extracted from the IP header.
- **Layer 4 – TCP / UDP:** source and destination ports are used to detect port scans (many unique destination ports) and repeated connection attempts to the same service.
- **Traffic analysis:** packet rate (packets/second) and bandwidth (bytes/second) are computed over sliding and per-second windows; protocol distribution and top talkers summarise who is using the network and how.
- **Security concepts:** port scanning, ARP cache poisoning, rogue/unknown devices, and connection flooding are modelled as detectable patterns, each treated as possible (not proven) malicious activity.

3.4 Detection Rules and Thresholds

Every threshold is read from config.json (not hard-coded), so behaviour can be tuned without changing code. Default values are shown below.

Detector	Technique	Default threshold	Severity
Port scan	Unique destination ports per source in a sliding window	≥ 20 ports / 10 s	High
Traffic anomaly	Packet count per source in a sliding window	> 200 pkts / 5 s	Medium
ARP anomaly	Change in the IP→MAC binding first seen for a host	Any MAC change	High
Unknown device	Source IP (and MAC) absent from the known-device inventory	First sighting	Medium
Repeated connections	Attempts to the same (src, dst, proto, dst-port) in a window	≥ 10 / 30 s	Medium
Deduplication	Suppress the same ongoing event	60 s quiet window	—

Table 1: Detection rules, techniques, default thresholds, and severities.

3.5 Tools and Technologies

- **Python 3.12** — core implementation language.

- **Scapy** — live packet capture and parsing (used only in the capture/parser layer).
- **Flask** — backend and serving of the dashboard and live portal.
- **SQLite (sqlite3)** — lightweight persistence for alerts, devices, and traffic metrics.
- **HTML, CSS, vanilla JavaScript + inline SVG** — real-time dashboards and charts, with no external frameworks.
- **pytest** — automated testing. Wireshark can be used alongside the tool to independently verify captured traffic.

4. Implementation

The project is organised as a Python package (nids) with clearly separated concerns, so parsing, detection, storage, and presentation are independent and individually testable.

Module	Responsibility
models.py	PacketRecord / Alert / Device data classes (independent of Scapy, Flask, SQLite)
parser.py	Converts a Scapy packet into a PacketRecord and records its length in bytes
detectors/	One file per rule, all sharing a common Detector base class
engine.py	Runs each record through the detectors + deduplicator and the traffic meter
dedup.py	Suppresses duplicate alerts for an ongoing event
metrics.py	TrafficMeter: aggregates packets into per-second protocol/source buckets
db.py	SQLite schema and read/write helpers (alerts, devices, traffic, talkers)
api.py	Read-only Flask JSON API (create_app factory)
web/	Alerts dashboard (/) and live traffic portal (/live) with static assets
livefeed.py	Background synthetic feed for the --live-demo mode
capture.py	Scapy sniff() wrapper — the only place that performs live capture
config.py / config.json	Loads, validates, and stores all thresholds and the device inventory

Table 2: Project modules and their responsibilities.

4.1 Detection Engine

Each detector is a small object that inspects one record at a time, keeps its own in-memory sliding-window history, and returns an Alert or None. The engine feeds every record to all enabled detectors, deduplicates the alerts, and stores the survivors. Detectors never touch the database directly, which keeps the rules pure and testable.

4.2 Traffic Metering and the Live Portal

Every record that reaches the engine is also counted by the TrafficMeter. Packets are grouped into one-second buckets, separately per protocol and per source IP, recording both packet counts and byte totals. Completed seconds are flushed to the traffic and talkers tables; data older than the retention window (default 900 s) is pruned. The /api/traffic?window=N endpoint returns a per-second time series, a protocol breakdown, the top talkers, and a summary (totals plus peak/average packets- and bytes-per-second). The /live page polls this endpoint every two seconds and draws the bandwidth chart (inline SVG), protocol-mix bars, and a top-talkers table using vanilla JavaScript.

4.3 API Endpoints

Endpoint	Returns
GET /api/health	{ "status": "ok" } — liveness check
GET /api/alerts?limit=N	Most recent security alerts (newest first)
GET /api/devices	Devices observed on the network
GET /api/stats	Total alerts, counts by severity, tracked devices
GET /api/traffic?window=N	Per-second series, protocol breakdown, top talkers, and summary over N seconds

Table 3: Read-only JSON API endpoints.

4.4 Running the System

- **Dashboard only:** python run.py (serves whatever is already stored).
- **Live demo (recommended, no admin rights):** python run.py --live-demo — a synthetic feed drives the real engine so alerts and traffic appear within seconds.
- **Real capture:** sudo python run.py --capture — sniffs real packets off an interface (root is usually required on macOS/Linux).
- Then open **http://127.0.0.1:5001/** (alerts) and **http://127.0.0.1:5001/live** (live traffic portal).

5. Results / Outputs

The system was verified in two ways: an automated test suite and an end-to-end live-demo run with the real detection engine and traffic meter.

5.1 Automated Testing

The full pytest suite was executed from the project root. It builds the pipeline from synthetic packet records (so it is deterministic and needs no privileges) and uses temporary SQLite databases that never touch real data.

Result: 183 tests passed (0 failed). Coverage includes the parser, each detector, the dedup layer, the database, the traffic meter, the API (including the /api/traffic endpoint), and the web pages.

5.2 Detection Test Cases

Scenario (input)	Expected alert	Observed
One source contacts 20 distinct destination ports within 10 s	port_scan (High)	Raised
Burst of 250 packets from one source within 5 s	traffic_anomaly (Medium)	Raised
One IP seen with two different MAC addresses (ARP reply)	arp_anomaly (High)	Raised
Traffic from an IP not in the known-device inventory	unknown_device (Medium)	Raised
10 connection attempts to the same host:port within 30 s	repeated_connections (Medium)	Raised
Same ongoing event repeats within 60 s	Only one alert stored (rest suppressed)	Confirmed

Table 4: Detection test cases and observed results.

5.3 Live Traffic Analysis (Representative 60-second Window)

With the live-demo feed running, the /api/traffic endpoint reported the following over a 60-second window. The feed mixes ordinary TCP/UDP/ICMP traffic with occasional attack scenarios, so both normal bandwidth and anomalies are visible.

Metric	Observed value
Total packets (window)	287 packets
Total data (window)	299,616 bytes (≈ 292.6 KB)
Average throughput	≈ 4.9 KB/s (≈ 4.8 packets/s)
Peak throughput	≈ 155.9 KB/s (159,616 bytes in one second)
Distinct protocols	3 (TCP, UDP, ICMP)

Table 5: Traffic summary for a representative 60-second window.

Protocol mix: TCP — 267 packets, UDP — 12 packets, ICMP — 8 packets.

Top talker (source IP)	Bytes in window	Approx. share
10.0.0.2	268,560 B	90 %
10.0.0.11	23,472 B	8 %
10.0.0.12	6,800 B	2 %
10.0.0.13	784 B	< 1 %

Table 6: Top talkers by bytes in the sample window (synthetic demo addresses).

5.4 Screenshots

Insert screenshots of the running system here (captured while python run.py --live-demo is active):

- Figure 3: Live Traffic Portal (/live) — throughput cards, bandwidth-over-time chart, protocol-mix bars, and top-talkers table.
- Figure 4: Alerts Dashboard (/) — summary cards, recent alerts, and tracked devices.

[Paste Figure 3 and Figure 4 screenshots here]

5.5 Analysis

- The detectors reliably flag their target scenarios, and the deduplication layer prevents alert floods for a single ongoing event, so the dashboard stays readable.
- Bandwidth and protocol views clearly distinguish ordinary traffic from attack bursts — for example, a 250-packet burst produces a visible spike in the per-second bandwidth chart and simultaneously triggers the traffic-anomaly detector.
- Because every threshold lives in config.json, sensitivity can be tuned to reduce false positives without code changes.
- Detections are reported as possible suspicious activity, not as proof of an attack, which reflects real-world IDS practice.

6. Conclusion

The project successfully implements a lightweight, transparent Network Intrusion Detection and Traffic Monitoring System that captures and parses packets, detects five classes of possible suspicious activity, and measures live traffic — bandwidth, protocol distribution, and top talkers — presenting everything on a real-time, localhost-only web portal. It demonstrates core networking concepts in practice (Ethernet/ARP, IPv4, TCP/UDP/ICMP, ports, packet sizes, and sliding-window traffic analysis) rather than merely describing them, and it is validated by an automated suite of 183 passing tests and an end-to-end live demonstration.

Learning outcomes: packet capture and header parsing with Scapy, rule-based detection with sliding windows, deduplication, time-bucketed traffic aggregation, REST-style JSON APIs with Flask, SQLite persistence, and real-time visualisation with vanilla JavaScript and SVG.

Limitations: live capture on a normal (non-monitor-mode) host mostly sees traffic to and from that machine; the rules are threshold-based rather than learning-based; and the system is intentionally bound to localhost for safety.

Future scope: additional detectors (DNS tunnelling, SYN floods, DHCP spoofing), anomaly detection using statistical/ML baselines, alert notifications (email/ Telegram), export of captures to PCAP for Wireshark, and optional deployment behind authentication for multi-host monitoring.

7. References

- Scapy documentation — https://scapy.readthedocs.io/
- Flask documentation — https://flask.palletsprojects.com/
- Python sqlite3 module — https://docs.python.org/3/library/sqlite3.html
- RFC 826 — An Ethernet Address Resolution Protocol (ARP).
- RFC 793 — Transmission Control Protocol (TCP); RFC 768 — User Datagram Protocol (UDP).
- W. Stallings, Cryptography and Network Security; A. S. Tanenbaum, Computer Networks.